MANIPAL ACADEMY OF HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL INSTITUTE OF TECHNOLOGY

School of Computer Engineering

CES 2112 Digital Systems Lab

(Open Ended Experiment Report)

on

SMART VENDING MACHINE CONTROLLER USING DIGITAL LOGIC

Submitted by

Team member name	Section	Registration No.	Roll No.
Kanav Nathany	I	251090050446	13
Shailja Jhajharia	I	251090051214	28
Gauri Nikhil Chandak	I	251090051220	29
Lavanya Sandeep Borkar	I	251090050708	17
Jairaj Singh Mission	I	251090050235	9

Under the guidance of:

Dr Anoop B N, Assistant Professor (Senior Scale)

School of Computer Engineering, MIT Manipal

Date: 21/10/2026

TABLE OF CONTENTS

1. PROBLEM STATEMENT 3
1.1 Scope and assumptions 3
2. OBJECTIVES 4
2.1 Primary objective 4
2.2 Specific objectives 4
2.3 Sustainable Development Goals 4
3. SYSTEM DESIGN AND BLOCK DIAGRAM 4
3.1 Proposed architecture 4
3.2 Module description 5
3.3 Controller state machine 6
3.4 Sustainability features 6
3.5 Design decisions 7
4. VERILOG HDL IMPLEMENTATION 7
4.1 Currency validation and comparator (Kanav Nathany) 7
4.2 Transaction FSM with sleep/wake (Shailja Jhajharia) 8
4.3 Product selection and inventory (Gauri Nikhil Chandak) 9
4.4 Change and refund calculator (Lavanya Sandeep Borkar) 9
4.5 Sustainability monitor (Jairaj Singh Mission) 10
4.6 Top-level integration 10
4.7 Testbenches 11
5. SIMULATION, VERIFICATION AND RESULTS 11
5.1 Experimental setup 11
5.2 Unit test results 11
5.3 System test cases and outcomes 12
5.4 Simulation waveforms 12
5.5 Active-versus-sleep evaluation 13
5.6 Results and discussion 14
6. INDIVIDUAL CONTRIBUTION 14
Contribution statement 15
7. CONCLUSION AND FUTURE SCOPE 15
7.1 Conclusion 15
7.2 Limitations and future scope 15
REFERENCES 15

1. PROBLEM STATEMENT

A vending machine sells products without an attendant, so its controller alone decides when money is accepted, whether a product can be sold, when it is dispensed and how much is returned. A wrong decision costs someone money: a coin counted twice or not at all, a product dispensed twice, change computed from a stale amount, or payment kept for a product that is out of stock. These are timing and sequencing problems, which makes the controller a natural fit for synchronous digital logic built from comparators, counters, encoders, arithmetic units and a finite state machine (FSM).

A basic controller also has two blind spots that matter for sustainability. It stays fully active while nobody is using it, and it gives the operator no early warning that a product is running out or any record of what actually sells. Restocking then happens on a fixed visit schedule rather than on demand. This project addresses both through logic that can be built and verified in hardware: a sleep mode entered after inactivity, low-stock and out-of-stock flags, and sales and activity counters. The work is framed against UN Sustainable Development Goal 12 (Responsible Consumption and Production) and Goal 7 (Affordable and Clean Energy) [1].

The proposed solution is a synchronous controller written in Verilog-2001 [2], split into five modules: currency validation and payment comparison, the transaction FSM with sleep/wake, product selection and inventory, change/refund calculation, and a sustainability monitor. A top-level module connects them. The interfaces, naming and timing conventions shared by all five modules were fixed in a team specification before any module was written [3].

1.1 Scope and assumptions

- The design is purely digital and verified in simulation. Inputs are switch/button signals (`coin_insert`, `coin_denom`, `product_select`, `product_code`, `cancel_req`) assumed synchronous to `clk`; debouncing, coin sensors, motors and displays are outside the scope.
- Money is unsigned integer rupees. Accepted coins are ₹5, ₹10 and ₹20, and the inserted amount is capped at ₹200 (`MAX_AMOUNT`).
- Four products with codes 00–11 and prices ₹10, ₹15, ₹20 and ₹30. Each starts with 5 units; a product is low on stock at 2 units or fewer.
- The machine always has enough change (unlimited change float). The sleep mode is a logical FSM state; the clock is not gated and no power is measured.

TABLE I. SYSTEM REQUIREMENTS

ID	Requirement	Verification
R1	Accept only ₹5, ₹10 and ₹20 coins; reject an invalid code, a coin while disabled, and any coin that would exceed ₹200	CUR_T02–T06
R2	Count each coin insertion exactly once, however long the button is held	CUR_T04, SYS_T07
R3	Dispense exactly once, and only when a product is selected, in stock and paid for (amount ≥ price)	FSM_T08, SYS_T02, SYS_T09
R4	Return correct change after a sale; refund the full amount on cancel or out-of-stock	CHG_T02–T16, SYS_T03–T06
R5	Decrement stock once per sale, never below 0, and flag low and out-of-stock products	INV_T08–T11, SYS_T14
R6	Clear a transaction only after the return amount has been computed	FSM_T08, SYS_T16

ID	Requirement	Verification
R7	Sleep after inactivity only when no transaction is pending, and wake without losing the waking input	FSM_T02–T07, SYS_T10–T13
R8	Count sales and active/sleep cycles with saturating counters	SUS_T01–T11, SYS_T15, SYS_T20

2. OBJECTIVES

2.1 Primary objective

To design, implement and verify in synthesizable Verilog-2001 a vending machine controller that handles coin input, product selection, payment validation, dispensing, change and refund, and inventory tracking correctly in every transaction, and that adds idle-time sleep and stock/sales monitoring in support of SDG 12 and SDG 7.

2.2 Specific objectives

- Validate and accumulate coins with a decoder, a rising-edge detector and a 9-bit overflow-checked adder, and compare the amount with the price (comparator).
- Sequence every transaction with a six-state FSM that is the only source of control signals, including a sleep state entered after 32 idle cycles.
- Latch the selected product (encoder/decoder and price table), track per-product stock, and raise low-stock and out-of-stock flags.
- Compute change by subtraction protected against underflow, and a full refund on cancel or out-of-stock.
- Count completed sales (total and per product), active and sleep cycles, and wake-ups with saturating counters.
- Verify each module with its own self-checking testbench and the integrated system with 23 scenario tests, and document expected versus observed results.

2.3 Sustainable Development Goals

SDG 12 (primary): stock flags and per-product sales counts support planned restocking, and no payment is kept for a product that cannot be dispensed. **SDG 7 (secondary):** an idle-time sleep state, with active/sleep cycle counts as a digital proxy. These features support the goals; no energy or waste saving is claimed without measurement (Table III).

3. SYSTEM DESIGN AND BLOCK DIAGRAM

3.1 Proposed architecture

Fig. 1 shows the top-level module `vending_machine_top` and its five sub-modules. The FSM is the only control authority: it enables coin and product acceptance, starts dispensing and the return calculation, and issues the one-cycle `clear_txn` pulse that ends a transaction. The currency module owns the money accumulator, the inventory module owns the selected product, its price and the stock, and the change/refund module computes the amount to give back. The sustainability monitor only observes; it has no outputs that feed back into the control path, so it cannot interfere with a transaction. Every signal has exactly one driver. The design uses registered sequential control and state signals,

while combinational signals carry price, availability and payment status between modules. The interconnections were checked for combinational loops, and none were found (Section 5.1).

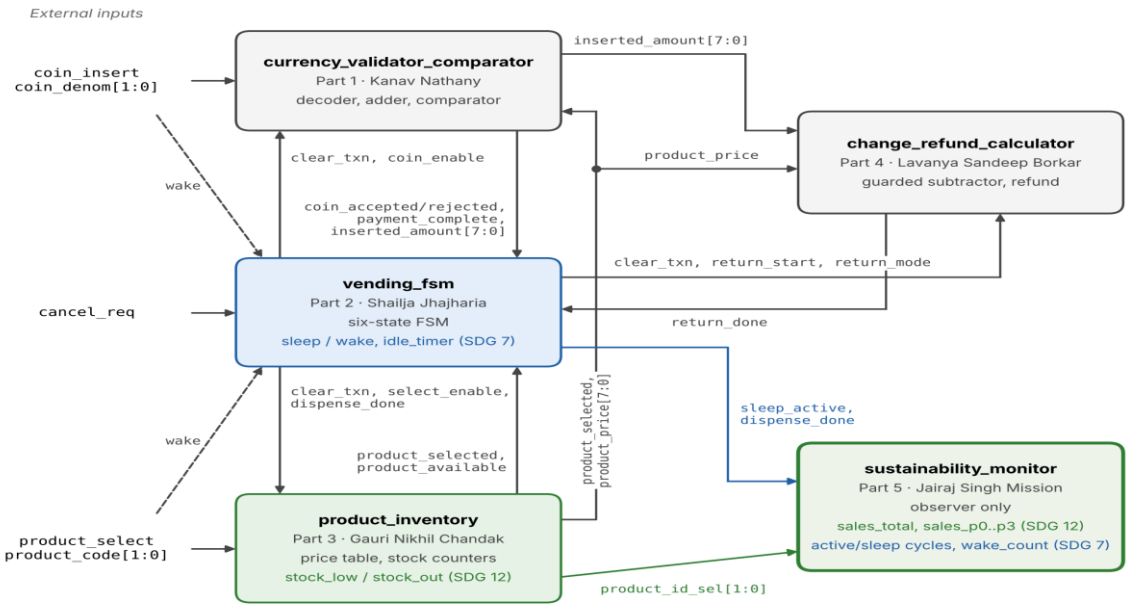


Fig. 1. Block diagram of vending_machine_top. Green marks SDG 12 features (stock flags, sales counters) and blue marks SDG 7 features (sleep/wake, activity counters).

In a purchase, the inventory module feeds `product_price` to the comparator, which raises `payment_complete` when the amount reaches the price. The FSM dispenses for four cycles, and its single `dispense_done` pulse decrements the stock and counts the sale. It then requests the return with `return_mode = 0` and clears the transaction after `return_done`. Cancel and out-of-stock use the same path with `return_mode = 1` (full refund, no dispense).

3.2 Module description

TABLE II. MODULE DESCRIPTION

Module (owner)	Main inputs	Main outputs	Function
`currency_validator_comparator` (Kanav Nathany)	coin inputs, `coin_enable`, `product_price`	`inserted_amount`, `coin_accepted/rejected`, `payment_*` flags	Decodes the coin, counts each insertion once, accumulates up to ₹200, compares amount with price
`vending_fsm` (Shailja Jhajharia)	raw inputs, `payment_complete`, `product_available`, `return_done`	`clear_txn`, enables, `dispense_*`, `return_start/mode`, `sleep_active`	Sequences the transaction; owns sleep/wake and the idle timer
`product_inventory` (Gauri Nikhil Chandak)	`product_select`, `product_code`, `dispense_done`	`product_price`, `product_available`, `stock_low`, `stock_out`	Latches the selection, looks up the price, keeps per-product stock and flags
`change_refund_calculator` (Lavanya Sandeep Borkar)	`return_start`, `return_mode`, amount, price	`return_amount`, `return_done`, `return_valid`, `return_error`	Change = amount − price (guarded);

Module (owner)	Main inputs	Main outputs	Function
			refund = full amount
`sustainability_monitor` (Jairaj Singh Mission)	`sleep_active`, `dispense_done`, `product_id_sel`	`sales_total`, `sales_p0..p3`, `active_cycles`, `sleep_cycles`, `wake_count`	Saturating counters for sales and activity; observer only
`vending_machine_top` (integration)	`clk`, `rst`, user inputs	all of the above as top-level ports	Instantiates and wires the five modules

3.3 Controller state machine

The FSM has six states in plain 3-bit binary encoding (Fig. 2). The sleep state is entered only from idle, and only when no money is inserted and no product is selected, so it can never interrupt a transaction. Coins and selections stay enabled during sleep, which means the input that wakes the machine is also processed normally rather than lost (SYS_T11 for a coin, SYS_T12 for a selection). In the collect state, a cancel takes priority over an out-of-stock product, which in turn takes priority over payment completion, so a customer who cancels is always refunded and never charged.

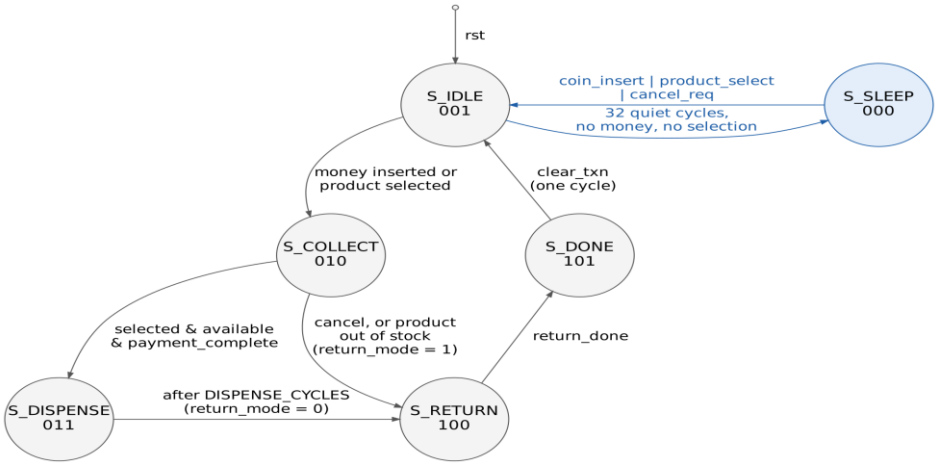


Fig. 2. State diagram of vending_fsm. Codes are the values of fsm_state; blue transitions belong to the sleep/wake feature.

3.4 Sustainability features

TABLE III. SUSTAINABILITY FEATURES AND THEIR LIMITS

Feature	Logic	SDG	What it demonstrates (and does not)
Sleep/wake (FSM)	16-bit idle timer; S_SLEEP after 32 quiet cycles; wake on any input	7	Controller changes behaviour when idle. No clock gating or power measurement, so no energy figure is claimed.
Active/sleep counters (monitor)	32-bit saturating cycle counters, 16-bit wake counter	7	Share of simulated cycles spent asleep, a digital proxy only.
Low/out-of-stock flags (inventory)	Comparators on each 8-bit stock counter	12	Restocking need is signalled before and at depletion. Waste reduction needs real usage data.

Feature	Logic	SDG	What it demonstrates (and does not)
Sales counters (monitor)	16-bit total and 8-bit per-product counters on `dispense_done`	12	Demand data per product is available for stock planning.
Refund when unavailable (FSM, inventory, change)	Out-of-stock path with `return_mode = 1`	12	No payment is kept for a product that cannot be dispensed.

3.5 Design decisions

- **Clocking and reset.** One clock and a synchronous, active-high reset in every module, with `if (rst)` as the first branch of each clocked block. The only registers without a reset branch are the three raw-button edge detectors (`coin_insert_prev`, `product_select_prev`, `cancel_req_prev`). Each copies its button on every clock edge, including while `rst` is high, so a button held high through reset release produces no rising edge and no event (CUR_T12, INV_T16). For `cancel_req` the FSM also guarantees this, because it leaves reset in S_IDLE and acts on cancel only in S_COLLECT (FSM_T19).
- **Registered handshakes.** Every pulse between modules (`clear_txn`, `dispense_done`, `return_start`, `return_done`, `coin_accepted`, `coin_rejected`) is a registered signal exactly one clock cycle wide. The FSM uses the two-block style [4], [5] and registers each output from `state_next`, so outputs are glitch-free and line up with the state.
- **Arithmetic safety.** Coin addition uses a 9-bit intermediate checked against `MAX_AMOUNT` before it is stored; the change subtraction is used only when amount ≥ price; stock decrements stop at 0 and all monitor counters saturate instead of wrapping.
- **Event priority.** `rst` > `clear_txn` > the module's normal update, in every module. Edges of raw inputs that arrive while a module is disabled are not queued.
- **Parameters.** `MAX_AMOUNT` = 200, `SLEEP_TIMEOUT_CYCLES` = 32, `DISPENSE_CYCLES` = 4, `INIT_STOCK` = 5, `LOW_STOCK_THRESHOLD` = 2 and the four prices are Verilog parameters passed down from the top level, so they can be changed without editing the logic.

4. VERILOG HDL IMPLEMENTATION

All modules are synthesizable Verilog-2001 [2] with ANSI-style ports. Sequential logic uses `always @(posedge clk)` with non-blocking assignments, combinational logic uses `assign` or `always @(*)` with a default for every output, and every `case` has a `default` branch, so no latches are inferred. The listings below show the core logic of each module, with omitted lines marked `...`; the ports are summarised in Table II. The complete source of every module and testbench, and the simulator logs, are provided in the supplementary appendix submitted with this report.

4.1 Currency validation and comparator (Kanav Nathany)

`currency_decoder` maps the 2-bit code to ₹0/5/10/20. The accumulator detects a rising edge of `coin_insert` by comparing it with `coin_insert_prev`, forms `amount_sum` on 9 bits, and accepts the coin only if the code is valid, coins are enabled, payment is not yet complete and the sum is within `MAX_AMOUNT`. Any other insertion event pulses `coin_rejected`. `payment_comparator` derives the four payment flags combinationally from the registered amount, and all four flags are 0 when no product is selected.

Listing: currency_validator_comparator.v (insertion detection and accumulator)

```
    assign insert_event = coin_insert & ~coin_insert_prev;
    assign amount sum   = {1'b0, inserted amount} + {1'b0, coin value};
    assign within_limit = (amount_sum <= MAX_AMOUNT);
    assign accept en    = insert event & coin valid & coin enable & ~payment complete &
within_limit;

    always @(posedge clk) begin
        if (rst) begin
    ...
        end else begin
            coin accepted <= 1'b0;
            coin rejected <= 1'b0;
            if (clear txn) begin
                inserted amount <= 8'd0;
            end else if (accept en) begin
                inserted_amount <= amount_sum[7:0];
                coin accepted   <= 1'b1;
            end else if (insert_event) begin
                coin rejected   <= 1'b1;
            end
        end
    end
```

4.2 Transaction FSM with sleep/wake (Shailja Jhajharia)

The combinational block computes `state_next`, the next idle and dispense timer values, and the next `return_mode` and `err_out_of_stock`. The clocked block registers all of them and derives every output from `state_next`. As a result, `dispense_done` is high in the last of the four dispense cycles, `return_start` in the first cycle of S_RETURN, and `clear_txn` during the single S_DONE cycle. `coin_enable` is forced low in S_COLLECT when the latched product is unavailable, so no further money is taken for it.

Listing: vending_fsm.v (idle/collect next-state logic and registered outputs)

```
            S_IDLE: begin
                if (txn pending) begin
                    state_next = S_COLLECT;
                end else if (raw activity) begin
                    idle timer next = 16'd0;
                end else if (idle timer >= SLEEP TIMEOUT CYCLES - 1) begin
                    state_next = S_SLEEP;
                end else begin
                    idle_timer_next = idle_timer + 16'd1;
                end
            end

            S_COLLECT: begin
                if (cancel event) begin
                    state_next       = S_RETURN;
                    return mode next = 1'b1;
                end else if (product unavailable) begin
                    state next            = S RETURN;
                    return mode next      = 1'b1;
                    err out of stock next = 1'b1;
                end else if (product selected & product available & payment complete) begin
                    state_next = S_DISPENSE;
                end
            end
    ...
    always @(posedge clk) begin
        if (rst) begin
    ...
        end else begin
            state            <= state_next;
            idle timer       <= idle timer next;
            dispense_timer   <= dispense_timer_next;
            return mode      <= return mode next;
            err_out_of_stock <= err_out_of_stock_next;
            clear txn        <= (state next == S DONE);
            coin_enable      <= coin_open_next;
```

```
            select_enable    <= (state_next == S_SLEEP) | (state_next == S_IDLE) | (state_next ==
S_COLLECT);
            dispense_active  <= (state_next == S_DISPENSE);
            dispense_done    <= (state_next == S_DISPENSE) & (dispense_timer_next == DISPENSE_CYCLES
- 1);
            return_start     <= (state_next == S_RETURN) & (state != S_RETURN);
            sleep_active     <= (state_next == S_SLEEP);
        end
    end
```

4.3 Product selection and inventory (Gauri Nikhil Chandak)

A selection event is latched only when selection is enabled and no product is held, which makes the choice sticky until `clear_txn`. The price comes from a four-entry `case` on the latched code (a decoder) and is 0 when nothing is selected. Stock is kept in a 4 × 8-bit register array that is decremented on `dispense_done` and clamped at 0. The latch and the stock are in separate blocks, so a clear and a dispense on the same edge both take effect. `stock_low` and `stock_out` are mutually exclusive by construction.

Listing: product_inventory.v (stock counters and flags)

```
    always @(posedge clk) begin
        if (rst) begin
            stock_count[0] <= INIT_STOCK;
            stock_count[1] <= INIT_STOCK;
            stock_count[2] <= INIT_STOCK;
            stock_count[3] <= INIT_STOCK;
        end else if (dispense_done & product_selected) begin
            if (stock_count[product_id_sel] != 8'd0)
                stock_count[product_id_sel] <= stock_count[product_id_sel] - 8'd1;
        end
    end
    ...
    assign stock_sel         = stock_count[product_id_sel];
    assign product_available = product_selected & (stock_sel != 8'd0);
    assign stock_out[0] = (stock_count[0] == 8'd0);
    assign stock_low[0] = (stock_count[0] != 8'd0) & (stock_count[0] <= LOW_STOCK_THRESHOLD);
```

4.4 Change and refund calculator (Lavanya Sandeep Borkar)

On `return_start` the module latches either `inserted_amount - product_price` (change mode) or `inserted_amount` (refund mode) and pulses `return_done`. Because both operands are unsigned, an amount below the price would wrap around (10 − 15 gives 251), so the subtraction is used only when `payment_sufficient` is true. Otherwise the result is 0 and `return_error` is set. `return_amount` is held after `clear_txn` so it can still be displayed; only `return_valid` drops. The listing omits the original header comments.

Listing: change_refund_calculator.v (guard and clocked block, reset branch omitted)

```
    assign payment_sufficient = (inserted_amount >= product_price);
    assign change_value = inserted_amount - product_price;

    always @(posedge clk) begin
        if (rst) begin
    ...
        end else begin
            return_done <= 1'b0;

            if (clear_txn) begin
                return_valid <= 1'b0;
            end else if (return_start) begin
                return_done  <= 1'b1;
                return_valid <= 1'b1;

                if (return_mode) begin
                    return_amount <= inserted_amount;
                    return_error  <= 1'b0;
```

```
                end else if (payment sufficient) begin
                    return_amount <= change_value;
                    return error  <= 1'b0;
                end else begin
                    return amount <= 8'd0;
                    return error  <= 1'b1;
                end
            end
        end
    end
```

4.5 Sustainability monitor (Jairaj Singh Mission)

All counters update in one clocked block. In each non-reset cycle exactly one of `active_cycles` and `sleep_cycles` increments, so their sum always equals the number of cycles since reset. A wake-up is detected as a falling edge of `sleep_active` using a one-cycle delayed copy. Each `dispense_done` pulse increments `sales_total` and the counter of the product given by `product_id_sel`. Every counter checks for all-ones before incrementing, so it saturates instead of wrapping.

Listing: sustainability_monitor.v (counter updates, reset branch omitted)

```
    always @(posedge clk) begin
        if (rst) begin
    ...
        end else begin
            sleep_active_prev <= sleep_active;

            if (sleep_active) begin
                if (sleep_cycles != 32'hFFFF_FFFF)
                    sleep_cycles <= sleep_cycles + 32'd1;
            end else begin
                if (active_cycles != 32'hFFFF_FFFF)
                    active_cycles <= active_cycles + 32'd1;
            end

            if (wake_event && (wake_count != 16'hFFFF))
                wake_count <= wake_count + 16'd1;

            if (dispense_done) begin
                if (sales_total != 16'hFFFF)
                    sales_total <= sales_total + 16'd1;
                case (product_id_sel)
                    2'b00:   if (sales_p0 != 8'hFF) sales_p0 <= sales_p0 + 8'd1;
                    2'b01:   if (sales_p1 != 8'hFF) sales_p1 <= sales_p1 + 8'd1;
                    2'b10:   if (sales_p2 != 8'hFF) sales_p2 <= sales_p2 + 8'd1;
                    2'b11:   if (sales_p3 != 8'hFF) sales_p3 <= sales_p3 + 8'd1;
                    default: sales_p0 <= sales_p0;
                endcase
            end
        end
    end
```

4.6 Top-level integration

`vending_machine_top` contains no logic of its own. Its parameters are passed down to the sub-modules, and its ports are the user inputs plus the outputs listed in Table II. Thirteen internal nets (`clear_txn`, `coin_enable`, `select_enable`, `dispense_done`, `return_start`, `return_mode`, `return_done`, `product_price`, `product_available` and the four payment flags) join the modules, and each of the five instances (`u_currency`, `u_fsm`, `u_product`, `u_change`, `u_sustain`) connects every port to the net of the same name (full listing in the supplementary appendix).

4.7 Testbenches

Every module has a self-checking testbench, plus one system testbench. Inputs change just after a falling edge and outputs are checked at the next falling edge, away from the sampling edge, and comparisons

use === so an unknown (x) never passes. Each check prints PASS or FAIL with its test ID, and each run ends with a SUMMARY line. The FSM testbench uses stubs for its neighbours. The system testbench also runs whole-simulation monitors that flag any pulse wider than one cycle, any clear_txn that does not follow return_done by exactly one cycle, any cycle with coins enabled during dispensing, and any cycle spent asleep with money inserted or a product selected.

5. SIMULATION, VERIFICATION AND RESULTS

5.1 Experimental setup

TABLE IV. SIMULATION SETUP

Item	Details
HDL / language	Verilog-2001 (IEEE 1364-2001) [2]
Simulator / version	Icarus Verilog 12.0, compiled with `iverilog -g2001 -Wall` (no warnings) [6]
Waveforms	VCD files written by every testbench; figures in this section are drawn from those VCD files
Synthesis / structural check	Yosys 0.70 [7] used only for a structural check of the flattened design (`check -assert`, `scc`): 0 problems, 0 combinational loops. No area, timing or FPGA results are reported
Test cases	6 testbenches, 289 self-checking checks (40 + 55 + 35 + 65 + 31 + 63), 0 failures
Clock	10 ns period in simulation; `SLEEP_TIMEOUT_CYCLES` = 32 (8 in the FSM unit test), `DISPENSE_CYCLES` = 4

5.2 Unit test results

TABLE V. UNIT AND SYSTEM TEST SUMMARY (OBSERVED FROM SIMULATOR LOGS)

Testbench	Test IDs	Checks	Passed	Failed	Notable checks
`tb_currency_validator_comparator`	CUR_T01–T16	40	40	0	₹200 limit; held button and button held through reset; exhaustive comparator sweep (131,072 cases)
`tb_vending_fsm`	FSM_T01–T19	55	55	0	Exact sleep timeout; handshake order; cancel priority; cancel held through reset; regression FSM_T18
`tb_product_inventory`	INV_T01–T16	35	35	0	Sticky selection; low/out thresholds; clamp at 0; select held through reset
`tb_change_refund_calculator`	CHG_T01–T16	65	65	0	Underflow guard; clear priority; reference sweep (5,408 cases)

Testbench	Test IDs	Checks	Passed	Failed	Notable checks
tb_sustainability_monitor	SUS_T01–T11	31	31	0	Saturation of all counters; active + sleep = cycles
tb_vending_machine_top	SYS_T01–T23	63	63	0	End-to-end scenarios and whole-run monitors

5.3 System test cases and outcomes

TABLE VI. EXPECTED VERSUS OBSERVED RESULTS FOR SYSTEM TESTS

Test ID	Input / condition	Expected output	Observed result
SYS_T02	Select P1 (₹15), insert ₹5 + ₹10	One dispense, change ₹0, P1 stock 5 → 4, sales 1	As expected. PASS
SYS_T03	Select P3 (₹30), insert ₹20 + ₹20	Change ₹10	Change ₹10. PASS
SYS_T04	Insert ₹20 + ₹20 first, then select P1	Change ₹25	Change ₹25. PASS
SYS_T05	Insert ₹10, then select out-of-stock P3	No dispense, refund ₹10, err_out_of_stock	Refund ₹10, flag raised, stock and sales unchanged. PASS
SYS_T10	No activity after a transaction	S_SLEEP after 32 cycles	32 cycles. PASS
SYS_T11	Insert ₹10 while asleep	Coin counted, FSM to S_COLLECT, wake_count + 1	As expected. PASS
SYS_T18	Reset mid-transaction	All outputs cleared, stock reloaded to 5	As expected. PASS
SYS_T19	Cancel on the same edge payment completes	Refund path wins	Refund ₹10, no sale. PASS
SYS_T20	Whole run	active_cycles + sleep_cycles = non-reset cycles	420 = 420. PASS
SYS_T22	Whole-run monitors (incl. SYS_T16)	clear_txn exactly one cycle after return_done, no wide pulses, no coins while dispensing, no sleep mid-transaction, no return_error	0 in every monitor. PASS

5.4 Simulation waveforms

Fig. 3 shows SYS_T03 from the system VCD (fsm_state codes as in Fig. 2). Product 3 is latched at 165 ns and the FSM enters S_COLLECT. Two ₹20 coins raise inserted_amount to 40, and at the next edge the FSM enters S_DISPENSE, dropping coin_enable. dispense_active stays high for four cycles, and the single dispense_done pulse in the last of them decrements stock_count[3] from 5 to 4 and increments sales_total. In S_RETURN, return_start is followed one cycle later by return_done with return_amount = 10. clear_txn then clears the amount and the selection, while return_amount is held for display.

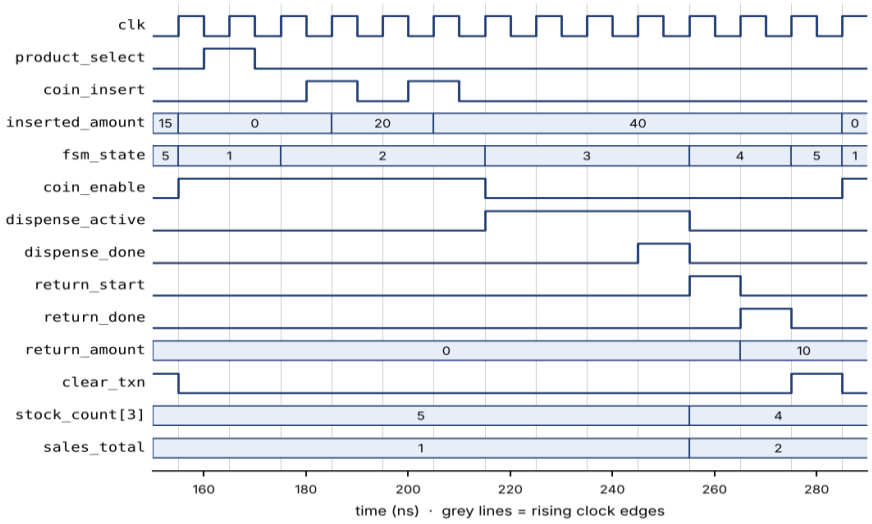


Fig. 3. SYS_T03: complete purchase of product 3 (₹30) with ₹40 inserted and ₹10 change.

Fig. 4 shows SYS_T10–T11. After 32 idle cycles the FSM is in S_SLEEP and `sleep_cycles` is counting. A ₹10 coin then arrives: because coins stay enabled in sleep, it is accepted on the same edge that wakes the FSM, and `wake_count` increments when `sleep_active` falls. The FSM then moves on to S_COLLECT with the coin already counted.

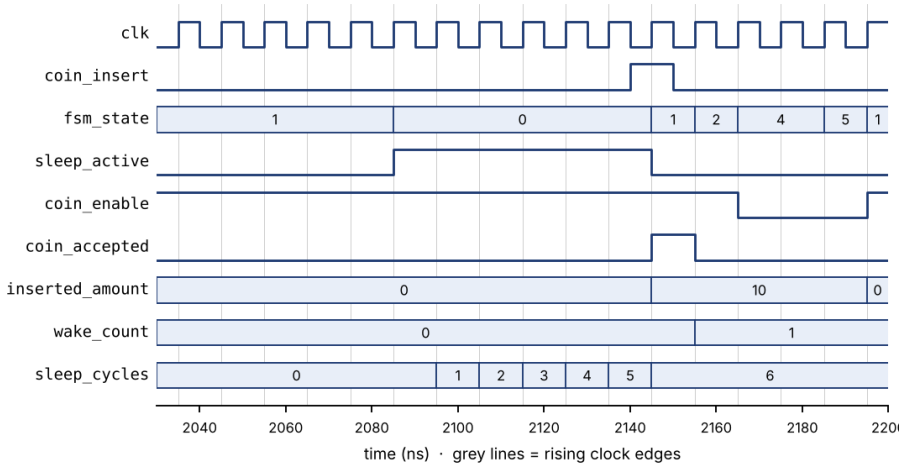


Fig. 4. SYS_T10–T11: sleep entry and wake-up by a coin that is counted, not lost.

5.5 Active-versus-sleep evaluation

SYS_T23 resets the controller and then runs five purchases of product 0, each preceded by a 300-cycle idle gap, followed by a final 300-cycle gap. From that reset the monitor counted 242 active cycles and 1,613 sleep cycles out of 1,855, so the controller was asleep for 86.9% of the simulated cycles, with 5 wake-ups and 5 sales (Fig. 5). This figure depends entirely on the chosen idle gaps and describes simulated clock cycles in a logical state. It is not a measured energy saving, because the clock is not gated and no power is modelled.

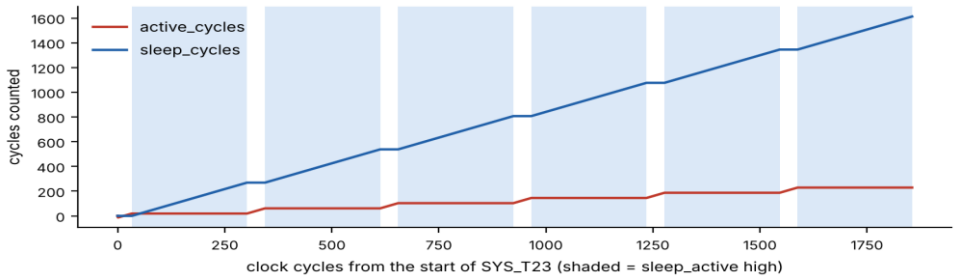


Fig. 5. SYS_T23: cumulative active and sleep cycles over a usage profile with 300-cycle idle gaps (shaded = sleep_active).

5.6 Results and discussion

All 289 checks pass, and every observed value in Tables V and VI is taken from the simulator logs. The return handshake was checked in every system transaction: `return_amount` is captured on the `return_done` pulse and compared with the expected value, each transaction has exactly one `return_done` and one `clear_txn`, `clear_txn` follows `return_done` by exactly one cycle (SYS_T22e), and `return_amount` is still readable after the clear. This covers exact payment (SYS_T02, T04b, T08b, T17a), overpayment (SYS_T03, T04c, T09a, T12b, T17b, T18e), cancel (SYS_T06a, T07b, T11d, T13b, T19b) and out-of-stock (SYS_T05a, T05e).

Integration testing (SYS_T05) found one defect: a coin inserted in the first idle cycle after a product's last unit was sold was rejected, because the FSM gated `coin_enable` with "latched product unavailable" in every state, including S_DONE. The gate now applies only in S_COLLECT; regression test FSM_T18 fails on the old code and passes on the fix. Deliberately broken versions of the modules and a miswired top level were each caught by at least one failing check.

6. INDIVIDUAL CONTRIBUTION

TABLE VII. INDIVIDUAL CONTRIBUTION OF TEAM MEMBERS

Team member	Section / Reg. No. / Roll No.	Individual tasks performed	Evidence / deliverables
Kanav Nathany	Section I 251090050446 Roll No. 13	Part 1. Designed `currency_validator_comparator`: coin decoder, rising-edge insertion detection, 9-bit accumulator with the ₹200 limit and payment comparator. Wrote its testbench (CUR_T01–T16) and Section 4.1.	Module, testbench and log (40/40 pass)
Shailja Jhajharia	Section I 251090051214 Roll No. 28	Part 2. Designed `vending_fsm` with the sleep/wake logic and idle timer. Integrated all modules in `vending_machine_top` and wrote the system testbench (SYS_T01–T23). Diagnosed and fixed the coin-enable defect found in SYS_T05. Wrote Sections 1–3, 4.2 and 4.6.	FSM (55/55) and system (63/63) testbenches and logs; Fig. 1–4
Gauri Nikhil Chandak	Section I 251090051220 Roll No. 29	Part 3. Designed `product_inventory`: selection latch, price decoder, per-product stock counters, and low- and out-of-stock flags. Wrote its testbench (INV_T01–T16) and Section 4.3.	Module, testbench and log (35/35 pass)
Lavanya Sandeep Borkar	Section I 251090050708 Roll No. 17	Part 4. Designed `change_refund_calculator` with the underflow guard, the return handshake and the `return_error` flag, and	Module, testbench and log (65/65 pass)

Team member	Section / Reg. No. / Roll No.	Individual tasks performed	Evidence / deliverables
		proposed the interface decisions adopted for integration. Wrote its testbench (CHG_T01–T16), including the 5,408-case reference sweep, and Section 4.4.	
Jairaj Singh Mission	Section I 251090050235 Roll No. 9	Part 5. Designed `sustainability_monitor` with saturating sales, per-product, activity and wake counters. Wrote its testbench (SUS_T01–T11), the SDG framing and the active-versus-sleep evaluation (SYS_T23), and Sections 2.3, 3.4, 4.5 and 5.5.	Module, testbench and log (31/31 pass); Fig. 5

Contribution statement

The team first agreed a shared interface specification [3]. Each member then designed and unit-tested their own module against it, Shailja Jhajharia integrated the modules and ran the system tests, interface questions were settled jointly, and all members reviewed the report.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

A smart vending machine controller was designed in Verilog-2001 as five independently owned modules. It validates coins, latches a product and its price, dispenses exactly once when paid, returns correct change, refunds in full on cancel or out-of-stock, and flags low and empty stock. For SDG 12 and SDG 7 it adds a sleep mode and saturating sales and activity counters. All 289 self-checking checks pass, covering requirements R1–R8.

7.2 Limitations and future scope

- Run the design on an FPGA board with debounced, synchronised inputs, and add clock gating with power measurement to turn the sleep share into an energy figure.
- Model a finite change float and break `return_amount` down into ₹20/₹10/₹5 coins.
- Add a restock input, stock kept across resets, a timeout refund for abandoned transactions, and a display for the sales and stock flags.

REFERENCES

[1] United Nations General Assembly, "Transforming our world: the 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

[2] IEEE Standard Verilog Hardware Description Language, IEEE Std 1364-2001, 2001.

[3] Project team, "Smart Vending Machine Controller Using Digital Logic: Project-wide technical specification and integration guide," v0.1, internal document, MIT Manipal, Oct. 2026.

[4] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2013.

[5] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[6] S. Williams, "Icarus Verilog." Accessed: Oct. 20, 2026. [Online]. Available: https://steveicarus.github.io/iverilog/

[7] C. Wolf and YosysHQ, "Yosys Open SYnthesis Suite." Accessed: Oct. 20, 2026. [Online]. Available: https://yosyshq.net/yosys/